

高级自然语言处理 第一次作业

12312515 王洛源

Q1 (1) :

BPE的思想源自文本压缩算法，目的是将每个词独立的词级分词改进为更高效的子词（即我们背单词时可能用到的词根词缀）。其运作方式为先将语料中的每个词拆成字符序列，如将low拆成[l, o, w]，这里每个单词的末尾还会加上特殊的单词结束符，如，防止两个连续单词的末尾字符和开头字符被错误的统计。随后统计相邻字符对出现频率，找到最高的一对，合并成一个新的符号加入到词汇表中，并更新语料中所有此符号对替换为新符号。重复此过程直到达到预设的词汇量大小。其结果是一些常见的子词比如est, er会被保留，而一些罕见的词会被拆成多个子词的组合，这比单词级分词更高效，此外由于生成的子词单元具有语言学意义，BPE在帮助机器理解词语意思的方面效果也更好

这里我们通过一个简单的py代码来简要演示这一过程，代码文件为Q1-1.py。代码中预设了语料库为["low", "lowest", "newer", "wider", "lower", "tallest"]，经过七次合并过程后，得到了low, er, est三个常见子词，这也与我们的语言习惯吻合。具体子词合并过程见下图

```
Step 1 merge: ('l', 'o')
Step 2 merge: ('lo', 'w')
Step 3 merge: ('e', 'r')
Step 4 merge: ('er', '</w>')
Step 5 merge: ('e', 's')
Step 6 merge: ('es', 't')
Step 7 merge: ('est', '</w>')
```

Q1 (2) :

代码文件为Q1-2.py。本地运行后从给定数据集提取得到的纯文本语料为train_text.txt，训练的模型得到的BPE规则文件为bpe_codes.txt，应用BPE后得到的分词结果文件为tokenized_text.txt，其中的“@@”表示子词之间的连接符（表示这个子词和下一个子词是同一个词的组成部分）。

Q2:

完整代码见Q2_transformer_model.py。这里对于每个todo部分给出实现代码。

TODO1: PositionalEncoding层 给每个词添加位置相关向量

```
def forward(self, x):
    x = x + self.pe[:,x.size(1),:]
    return self.dropout(x)
```

TODO2: FeedForward层 映射到高维→ReLU→Dropout→映射回原维度

```
def forward(self, x):
    return self.fc2(self.drop(self.relu(self.fc1(x))))
```

TODO3: MultiHeadAttention层 应用理论公式:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})V$$

```
def forward(self, q, k, v, mask=None):
    B = q.size(0)

    Q = self.q_linear(q).view(B, -1, self.num_heads, self.d_k).transpose(1, 2)
    K = self.k_linear(k).view(B, -1, self.num_heads, self.d_k).transpose(1, 2)
    V = self.v_linear(v).view(B, -1, self.num_heads, self.d_k).transpose(1, 2)

    # TODO: Compute attention scores
    # Hint:
    # 1. Multiply Q and K^T, then scale by sqrt(d_k)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
    # 2. If mask is provided, use masked_fill to set ignored positions to -1e9
    if mask is not None:
        scores = scores.masked_fill(mask==0, -1e9)
    # 3. Apply softmax to get attention weights (sum = 1)
    attn = torch.softmax(scores, dim=-1)
    attn = self.drop(attn)

    # 输出
    out = torch.matmul(attn, V)
    out = out.transpose(1, 2).contiguous().view(B, -1, self.num_heads * self.d_k)
    return self.out_linear(out)
```

TODO4: Residual层 残差链接+LayerNorm

```
def forward(self, x, sublayer):
    return x + self.drop(sublayer(self.norm(x)))
```

TODO5: EncoderLayer层

```
def forward(self, x, mask):  
    x = self.res_layers[0](x, lambda x: self.self_attn(x,x,x,mask))  
    x = self.res_layers[1](x, self.ffn)  
    return x
```

此外，为验证代码有效性，在代码最后添加了一段代码用于测试，经测试代码可以正常工作。这段测试用代码已被注释掉。

Q3:

见Q3.py文件